Complete Deep Learning Training and Prediction Guide

Table of Contents

Training Methods

1. Supervised Learning

What it is: Training with labeled input-output pairs **Use cases:** Classification, regression, object detection

Steps:

1. Prepare labeled dataset (X, y)
2. Split data into train/validation/test sets
3. Define loss function (cross-entropy, MSE, etc.)
4. Initialize model weights
5. Forward pass → compute predictions
6. Compute loss between predictions and true labels
7. Backward pass → compute gradients
8. Update weights using optimizer
9. Repeat steps 5-8 for multiple epochs

2. Unsupervised Learning

What it is: Training without labels to find patterns **Use cases:** Clustering, dimensionality reduction, anomaly detection

Steps:

1. Prepare unlabeled dataset
2. Define objective function (reconstruction error, clustering loss)
3. Initialize model
4. Forward pass through model
5. Compute unsupervised loss
6. Backward pass and weight updates

7. Evaluate using intrinsic metrics (silhouette score, reconstruction error)

3. Semi-Supervised Learning

What it is: Combines labeled and unlabeled data **Use cases:** When labeled data is scarce

Steps:

1. Prepare small labeled dataset + large unlabeled dataset
2. Train initial model on labeled data
3. Use model to generate pseudo-labels for unlabeled data
4. Combine original labels with high-confidence pseudo-labels
5. Retrain model on expanded dataset
6. Iterate steps 3-5

4. Self-Supervised Learning

What it is: Creates supervision signal from data itself **Use cases:** Pretraining, representation learning

Steps:

1. Design pretext task (masking, rotation, next token prediction)
2. Create pseudo-labels from data transformation
3. Train model to solve pretext task
4. Fine-tune on downstream task with real labels

5. Transfer Learning

What it is: Use pretrained model as starting point **Use cases:** Limited data, similar domains

Steps:

1. Load pretrained model
2. Freeze early layers (feature extraction)
3. Replace final layers for new task
4. Train only new layers initially
5. Optionally fine-tune entire model with lower learning rate

6. Few-Shot Learning

What it is: Learn from very few examples **Use cases:** Rare classes, rapid adaptation

Steps:

1. Train meta-model on many similar tasks

2. Present support set (few examples)
3. Model adapts quickly to new task
4. Evaluate on query set

Prediction Methods

1. Standard Inference

Steps:

1. Load trained model weights
2. Set model to evaluation mode (disable dropout/batch norm training)
3. Preprocess input data (same as training)
4. Forward pass through model
5. Apply activation function (softmax for classification)
6. Extract predictions

2. Batch Prediction

Steps:

1. Group multiple inputs into batches
2. Process entire batch simultaneously
3. More efficient than individual predictions
4. Useful for large datasets

3. Online/Streaming Prediction

Steps:

1. Process one sample at a time
2. Immediate prediction without batching
3. Useful for real-time applications
4. May require model optimization for speed

4. Ensemble Prediction

Steps:

1. Train multiple models (different architectures/hyperparameters)
2. Make predictions with each model
3. Combine predictions (voting, averaging, stacking)

4. Often improves accuracy and robustness

5. Test-Time Augmentation (TTA)

Steps:

1. Apply multiple transformations to input
2. Make predictions on each transformed version
3. Average or vote on predictions
4. Reduces variance, improves robustness

Hyperparameter Tuning

Core Hyperparameters and Their Effects

1. Learning Rate

What it does: Controls step size during weight updates

- **Too high:** Model diverges, unstable training
- **Too low:** Slow convergence, may get stuck in local minima
- **Typical range:** 1e-5 to 1e-1
- **Tuning methods:** Learning rate schedules, adaptive optimizers

2. Batch Size

What it does: Number of samples processed before weight update

- **Large batches:** Stable gradients, faster training, more memory
- **Small batches:** Noisy gradients, regularization effect, less memory
- **Typical range:** 16-512
- **Trade-offs:** Memory vs. gradient stability

3. Number of Epochs

What it does: How many times model sees entire dataset

- **Too few:** Underfitting
- **Too many:** Overfitting
- **Monitoring:** Use validation loss to determine optimal stopping point

4. Network Architecture

What it does: Defines model capacity and structure

- **Depth:** Number of layers (deeper = more complex patterns)
- **Width:** Number of neurons per layer (wider = more capacity)
- **Architecture type:** CNN, RNN, Transformer, etc.

5. Regularization Parameters

Dropout Rate:

- **What it does:** Randomly zeros neurons during training
- **Effect:** Prevents overfitting, improves generalization
- **Typical range:** 0.1-0.5

Weight Decay (L2 Regularization):

- **What it does:** Penalizes large weights
- **Effect:** Prevents overfitting, smoother decision boundaries
- **Typical range:** 1e-6 to 1e-2

6. Optimizer Parameters

Adam Parameters:

- **Beta1 (momentum):** 0.9 (typical), controls momentum
- **Beta2:** 0.999 (typical), controls adaptive learning rates
- **Epsilon:** 1e-8, numerical stability

SGD Parameters:

- **Momentum:** 0.9 (typical), accelerates convergence
- **Nesterov:** True/False, improved momentum variant

Hyperparameter Tuning Methods

1. Grid Search

Steps:

1. Define hyperparameter ranges
2. Create grid of all combinations
3. Train model for each combination
4. Select best performing combination **Pros:** Exhaustive, reproducible **Cons:** Exponentially expensive

2. Random Search

Steps:

1. Define hyperparameter distributions
2. Sample random combinations
3. Train models for sampled combinations
4. Select best performing **Pros:** More efficient than grid search **Cons:** May miss optimal combinations

3. Bayesian Optimization

Steps:

1. Define hyperparameter space
2. Build probabilistic model of objective function
3. Use acquisition function to select next hyperparameters
4. Train model and update probabilistic model
5. Repeat until convergence **Pros:** Very efficient, guided search **Cons:** More complex to implement

4. Evolutionary/Genetic Algorithms

Steps:

1. Initialize population of hyperparameter sets
2. Evaluate fitness (model performance)
3. Select best performers
4. Create offspring through crossover and mutation
5. Repeat for multiple generations

Step-by-Step Implementation Guide

Phase 1: Data Preparation

1. **Data Collection**
   - Gather relevant dataset
   - Ensure data quality and consistency
2. **Data Preprocessing**
   - Handle missing values
   - Normalize/standardize features
   - Encode categorical variables
3. **Data Splitting**
   - Training set (60-70%)
   - Validation set (15-20%)

- Test set (15-20%)

4. **Data Augmentation** (if applicable)
   - Increase dataset size artificially
   - Improve model generalization

Phase 2: Model Design

1. **Choose Architecture**
   - CNN for images
   - RNN/LSTM/Transformer for sequences
   - MLP for tabular data
2. **Define Model Structure**
   - Input layer dimensions
   - Hidden layer sizes and types
   - Output layer (classes for classification)
3. **Initialize Weights**
   - Xavier/Glorot initialization
   - He initialization for ReLU
   - Pretrained weights for transfer learning

Phase 3: Training Setup

1. **Choose Loss Function**
   - Classification: Cross-entropy, focal loss
   - Regression: MSE, MAE, Huber loss
   - Custom losses for specific tasks
2. **Select Optimizer**
   - Adam (good default)
   - SGD with momentum (for fine-tuning)
   - AdamW (Adam with weight decay)
3. **Set Initial Hyperparameters**
   - Learning rate: 1e-3 (good starting point)
   - Batch size: 32-128
   - Epochs: Start with 50-100

Phase 4: Training Process

1. **Training Loop**

```
for epoch in range(num_epochs):
    for batch in train_loader:
        # Forward pass
        predictions = model(batch.data)
        loss = loss_function(predictions, batch.labels)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Log metrics
        track_training_loss(loss)
```

2. **Validation**
    - Evaluate on validation set each epoch
    - Monitor for overfitting
    - Save best model weights
3. **Early Stopping**
    - Stop training when validation loss stops improving
    - Prevents overfitting
    - Saves computational resources

Phase 5: Hyperparameter Optimization

1. **Initial Baseline**
    - Train model with default hyperparameters
    - Establish baseline performance
2. **Learning Rate Tuning**
    - Use learning rate finder
    - Try range [1e-5, 1e-1]
    - Plot loss vs learning rate
3. **Architecture Tuning**
    - Vary number of layers
    - Adjust layer widths
    - Try different activation functions
4. **Regularization Tuning**
    - Adjust dropout rates
    - Tune weight decay

- Try different regularization techniques

5. **Advanced Tuning**
   - Use automated tools (Optuna, Ray Tune)
   - Multi-objective optimization
   - Consider computational budget

Phase 6: Model Evaluation

1. **Performance Metrics**
   - Classification: Accuracy, F1-score, ROC-AUC
   - Regression: RMSE, MAE, R^2
   - Custom metrics for specific domains
2. **Cross-Validation**
   - K-fold cross-validation
   - Stratified CV for imbalanced data
   - Time series CV for temporal data
3. **Test Set Evaluation**
   - Final evaluation on held-out test set
   - Report confidence intervals
   - Analyze failure cases

Phase 7: Production Deployment

1. **Model Optimization**
   - Quantization (reduce precision)
   - Pruning (remove unnecessary weights)
   - Knowledge distillation
2. **Inference Pipeline**
   - Input preprocessing
   - Model prediction
   - Output postprocessing
   - Error handling
3. **Monitoring**
   - Track prediction latency
   - Monitor data drift
   - Set up alerts for anomalies

Advanced Techniques

Learning Rate Scheduling

Types:

- **Step decay:** Reduce LR at fixed intervals
- **Exponential decay:** Exponential reduction
- **Cosine annealing:** Cosine function reduction
- **Warm restarts:** Periodic LR resets

Implementation:

- Start with higher LR for faster initial learning
- Reduce LR as training progresses for fine-tuning
- Use validation loss to guide schedule

Advanced Optimization

Techniques:

- **Gradient clipping:** Prevents exploding gradients
- **Learning rate warmup:** Gradually increase LR at start
- **Mixed precision training:** Use FP16 for memory efficiency
- **Gradient accumulation:** Simulate larger batch sizes

Regularization Techniques

Methods:

- **Dropout:** Random neuron deactivation
- **Batch normalization:** Normalize layer inputs
- **Layer normalization:** Alternative to batch norm
- **Data augmentation:** Artificial data expansion
- **Label smoothing:** Soften hard labels

Common Pitfalls and Solutions

Training Issues

Problem: Loss not decreasing **Solutions:**

- Check learning rate (too high/low)

- Verify data preprocessing
- Check for implementation bugs
- Try different initialization

Problem: Overfitting **Solutions:**

- Increase regularization
- Reduce model complexity
- Add more training data
- Use early stopping

Problem: Underfitting **Solutions:**

- Increase model capacity
- Reduce regularization
- Train for more epochs
- Check for data quality issues

Hyperparameter Tuning Tips

1. **Start simple:** Begin with basic hyperparameters
2. **One at a time:** Tune parameters systematically
3. **Use validation set:** Never tune on test set
4. **Consider interactions:** Some hyperparameters interact
5. **Budget time:** Set limits on tuning experiments
6. **Document everything:** Track all experiments

Practical Implementation Checklist

Before Training

- [ ] Data quality check completed
- [ ] Train/val/test splits created
- [ ] Baseline model established
- [ ] Evaluation metrics defined
- [ ] Computing resources allocated

During Training

- [ ] Training/validation loss monitored
- [ ] Learning curves plotted
- [ ] Checkpoints saved regularly

- [ ] Hyperparameters logged
- [ ] Resource usage monitored

After Training

- [ ] Test set evaluation completed
- [ ] Model performance analyzed
- [ ] Failure cases investigated
- [ ] Model artifacts saved
- [ ] Documentation updated

For Production

- [ ] Model optimized for inference
- [ ] Deployment pipeline tested
- [ ] Monitoring systems setup
- [ ] Rollback plan prepared
- [ ] Performance benchmarks established

This guide provides a comprehensive framework for deep learning projects. The key is to start simple, iterate systematically, and always validate your choices with proper evaluation metrics.